Shizuku Object Firmware Composition 設計仕様書

状態: 実装設計正本 日付: 2026-10-15 対象リポジトリ: `Shizuku-objfw` (`feat/object-firmware-composition`)

1. 背景と課題 (現状分析)

1.1 現行の固定 ID 生成の課題

現行のビルド構成 (`BUILD.bazel:12-28`) では、オブジェクト ID の一覧表 `generated_object_ids/shizuku/object_ids.hpp` が 単一の `genrule` (`name = "object_ids_hpp"`) によって生成されている。 生成には以下の 3 つのリストが静的に渡されている:

- `source/apps/objects.list`
- `source/selftest/objects.list`
- `//modules/pico_sdk_support:objects.list`

この構成には以下の重大な構造的制約が存在する:

1. 全再コンパイルの連鎖: `object_ids.hpp` は `internal_headers` (`BUILD.bazel:44-50`) に含まれ、カーネル、カーネルオブジェクト、全自己テスト、周辺ドライバがすべてこれを include している。オブジェクトを 1 つ追加・変更すると全 TU の再コンパイルが走る。
2. 即値埋め込みによるキャッシュ阻害: `tools/gen_object_ids.py:42` により `constexpr uintptr_t <name> = <id>;` として生成されるため、オブジェクトのオブジェクトファイル (`.o`) 内に ID 即値が直接埋め込まれ、合成 (composition) ごとに別個のコンパイルが必要となる。
3. 外部合成の不能: 外部リポジトリ (XNO や scratch スケッチ) から `bazel_dep` で利用する場合、固定の `objects.list` に含まれない独自のオブジェクトを動的に合成してファームウェアを作成することができない。
4. ビルド設定の外部流出: RP2350 ターゲットでは `.bazelrc` に書かれた `--platforms=@pico-sdk//bazel/platform:rp2350` や `--@pico-sdk//bazel/config:PICO_BOARD=pico2_w` など数十行の設定に依存しており、外部リポジトリが `.bazelrc` を複製しないとビルドできない。

1.2 確定前提・不変条件との照合

本設計では以下の不変条件を厳格に保持する:

- 不変条件 **1 (pico-sdk)**: pico-sdk は公式 Bazel (BCR 2.2.0) から CMake に戻し、SDK 内に閉じる。Shizuku は Bazel で SDK 非依存 `.a` を出力し、SDK CMake が最終リンクを行う。
- 不変条件 **2 (API 基本単位)**:
  - `shizuku_object(name, srcs, deps)`: firmware/ID 未定の `cc_library` + 型付き marker provider。合成非依存。
  - `shizuku_firmware(name, target, objects=[...])`: 合成時に ID / 親子 / 権限 / chip を決定し、単一 `.a` を生成して SDK CMake (IDF / pico-sdk) が最終リンク。
  - `shizuku_sketch(name, srcs)`: `setup()` / `loop()` を呼ぶ entry と 1 object を作る糖衣 + firmware 短縮形。
  - 利用者の `platform/board/PICO_*/C++23/-Og/-g3` は transition で macro 側固定、利用側 `.bazelrc` コピー不要。
- 不変条件 **3 (ID 解決)**: ID は合成時採番。object は名前ベース `extern const` symbol 参照、合成ごとの `.c` 定義をリンクで解決。同 chip/toolchain なら composition を越えて object の compile action を完全に再利用。
- 不変条件 **4 (RAM 配置)**: `chips/` の `SHIZUKU_RAM_TEXT` 等で `.iram1.*` セクションを明示し、SDK header 非依存とする。
- 不変条件 **5 (検証順序と権限)**: C6 縦貫 → 自レビュー → RP → 統合検証。親子関係を権限根拠とし、C ABI を維持する。

2. アーキテクチャ設計

2.1 シンボル解決と ID 採番 (合成非依存 compile action)

オブジェクトはヘッダ `shizuku/object_id.hpp` において名前ベースのシンボルを `extern const` として宣言する:

```
namespace shizuku::object_id {
  extern const uintptr_t my_obj;
}
```

shizuku_firmware の合成ルール (shizuku_firmware_composition) は、指定された objects のリストを受け取り、以下を行う:

1. 重複検証: オブジェクト名の重複を検査 (duplicate object error)。
2. 親子関係検証: 指定された親オブジェクトが存在するか検証 (missing/invalid parent error)。
3. 上限検査: 合成オブジェクト総数が上限 (既定 64) を超過していないか検査 (capacity exceeded error)。
4. シンボル定義コード生成:

   ```
   // 生成された合成テーブル: shizuku_composition_ids.c
   #include <stdint.h>
   const uintptr_t shizuku_object_id_root = 0;
   const uintptr_t shizuku_object_id_my_obj = 2;
   // ...
   ```

   また、C++ 名前空間エイリアス用の定義も提供する。

この方式により、各 shizuku_object のコンパイルアクションは合成時の ID 割当に全く影響されず、異なるファームウェア合成間で同一の .o をキャッシュから再利用できる。

2.2 Starlark Provider とルール定義 (defs.bzl)

ShizukuObjectInfo Provider

```
ShizukuObjectInfo = provider(
    doc = "Shizuku Object marker and metadata",
    fields = {
        "object_name": "Unique name of the object",
        "entry_point": "Entry function symbol (default: <name>_main)",
        "parent": "Parent object name or None",
        "privileged": "Boolean flag indicating privileged execution",
        "cc_info": "CcInfo provider containing compiled objects and headers",
    },
)
```

shizuku_object ルール / マクロ

```
def shizuku_object(name, srcs = [], hdrs = [], deps = [], parent = None, privileged = False,
**kwargs):
    # cc_library を生成し、ShizukuObjectInfo を付与したルールでラップ
    ...
```

shizuku_firmware マクロ / ルール

target 引数 (esp32c6, rp2350 等) を受け取り、以下を行う:

1. Incoming transition を適用し、ターゲットアーキテクチャのプラットフォーム、C++23、-Og、-g3 を強制設定。
2. 依存オブジェクト、カーネル、カーネルオブジェクトを結合して単一のアーカイブ (libshizuku_firmware.a) を生成。
3. ESP32-C6 の場合は modules/esp_idf_support/firmware/build.sh (または専用ラッパー) を通じて idf.py リンクを実行し、.bin を出力。
4. RP2350 の場合は pico-sdk CMake プロジェクト (常設 build dir を再利用) を通じて最終リンクを実行し、.uf2 を出力。

shizuku_sketch マクロ

```
def shizuku_sketch(name, srcs = [], deps = [], target = "esp32c6", **kwargs):
    # 1. setup() / loop() を呼び出す main entry を持つ shizuku_object を生成
    # 2. shizuku_firmware(name = name, target = target, objects = [":" + name + "_obj"], ...) を呼び出す
    ...
```

2.3 RAM 常駐 API (`SHIZUKU_RAM_TEXT`)

`chips/esp32_c6/chip.hpp` および `chips/rp2350/chip.hpp` において:

- ESP32-C6: `#define SHIZUKU_RAM_TEXT __attribute__((section(".iram1.text")))`
- RP2350: `#define SHIZUKU_RAM_TEXT __attribute__((section(".time_critical.text")))`
- ホストテスト / 共通: `#define SHIZUKU_RAM_TEXT`

SDK のヘッダ (`esp_attr.h` や `pico/platform.h`) を include することなく、リンカスクリプトのセクション配置ディレクティブのみで RAM 常駐を実現する。 実リンク後の ELF および `.map` ファイルにおいて、該当シンボルが `.iram1` (C6) または `.time_critical` (RP) に配置されていることを `nm` および `grep` で検証する。

3. 外部 Scratch 利用仕様 (目的 B)

外部 scratch リポジトリ (`/private/tmp/scratch_*`) においては、

- `MODULE.bazel`:

```
bazel_dep(name = "shizuku")
local_path_override(
    module_name = "shizuku",
    path = "/Users/ishigakiyua/github/Shizuku-objfw",
)
```

- `.bazelrc`: 作成不要 **(0 バイト / 存在しない)**
- `BUILD.bazel`:

```
load("@shizuku//:defs.bzl", "shizuku_sketch")

shizuku_sketch(
    name = "my_sketch",
    srcs = ["sketch.cpp"],
)
```

これらにより、コマンド `bazel build //:my_sketch_archive` だけで、 一切の `.bazelrc` 追加設定なしに実バイナリ (.a) が生成される。

4. 実装確定方針と境界契約 (2026-10-15 追記)

4.1 Canonical C ABI シンボルと唯一実体

ID 定義の実体は、合成ごとに生成される C ソース (`shizuku_composition_ids.c`) 内に `const uintptr_t shizuku_object_id_<name> = <id>;` として 1 回のみ配置される。C++ 側の `shizuku::object_id::<name>` は初期化不要な `inline constexpr const uintptr_t&` 参照エイリアスとして解決し、動的初期化 (static ctor) を完全に排除する。グローバルな ID 生成 (`object_ids.hpp` の単一 genrule) は段階的に廃止し、自己テストおよび apps も個別の `shizuku_object` へ移行する。

4.2 Firmware Manifest Ownership と Runtime ROOT_OBJECT 分離

オブジェクトの親子関係 (`parents`) および特権フラグ (`privileged_names`) は、`shizuku_object` ではなく `shizuku_firmware` が一元的に所有し、静的な検証 (循環参照・自己親・存在しない親の fail-closed 検査) を行う。 ランタイムの初期入力テーブル (`shizuku_composition_manifest[]`) において、ルート親の ID はランタイムの `ROOT_OBJECT = 1` (ブートスレッド・root ハンドラ) を基準とし、レガシーな C ABI シンボル `shizuku_object_id_root = 0` (KERNEL_OBJECT) とは明確に分離する。オブジェクト上限は予約スロット 2 つ (0, 1) を引いた 62 個とする。

4.3 スケッチ実行規約と Yield 境界

`shizuku_sketch` / `shizuku_sketch_object` は、単独ファームウェア生成だけでなく provider 付き object として複数オブジェクト合成に渡すことができる。エントリラッパー関数は `<name>_main()` の固有識別子を用い、`setup()` を 1 度呼び出した後 `loop()` を無限反復する。現行カーネルには純粋な C ABI の yield 関数 (例: `shizuku_yield`) は未実装であり、実在する API は C++ の `shizuku::services::yield()` のみであるため、架空の C シンボルを定義せず、C ABI yield 接続は未達・次期課題として扱う。

4.4 Transition Flags と外部 Scratch

shizuku_firmware は incoming transition により、platforms=@shizuku//toolchains/rv32_esp:esp32c6、board=esp32_c6、-std=c++23、-Og、-g3、-DSHIZUKU_C6_SKELETON_PERMITTED=1、strip=never を強制する。トップレベルのバイナリ生成ルールに transition を持たせることで、ホスト側 (macOS arm64 等) のデフォルト設定による incompatibility skip を防止する。IDF 環境変数 (IDF_PATH, IDF_TOOLS_PATH) は Starlark rule attr または repository_rule 経由での注入が望ましいが、現状は Bazel の --action_env 依存が残っており、今後の外部モジュール完全透過化における論点である。

4.5 SDK Builddir と set-target 注意点

build.sh において第 5 引数または環境変数 SHIZUKU_IDF_BUILD_DIR による常設 builddir の再利用を可能としたが、ESP-IDF の idf.py set-target は既存の CMakeCache.txt をクリア・再構成する挙動を示すため、キャッシュ無効化を防ぐには既存 config との照合による set-target スキップが必要となる。また、caller (Bazel 側) から第 5 引数が未接続の経路では毎回 mktemp -d が使われるため、完全な incremental link の主張は行わず部分対応にとどめる。

4.6 現状の到達点と未達事項

- 到達: Starlark provider、shizuku_object、shizuku_firmware (C6 transition, manifest ownership, limit 62, 単一 .a マージ)、外部 scratch での .bazelrc なしアーカイブ生成・nm シンボル検証。
- 未達: C6 実機 .bin の最終リンクおよび runtime CREATE_OBJECT ハンドラ連携 (ID 担当による root BUILD / templates 移行完了後の協調待ち)。